Module 7：ML 前導與學習路線

課程總時長： 3 小時（本課程最終模組） 模組定位：
在六個模組建立的資料分析底盤之上，打開前方三條技術縱深的視野，並以五條職涯路線為學員指出離開課程後

> "Machine learning turns patterns into prediction."

一、模組定位

這是一門 24 小時課程的最後一塊拼圖。前六個模組建立了：

- M0 開場：課程全景與學習心法
- M1 資料思維：從問題出發看資料
- M2 Python 基礎：型別、語法、資料結構
- M3 NumPy：數值運算與 tensor 概念
- M4 pandas：表格處理與 EDA
- M5 視覺化：從數字到洞察
- M6 統計：推論思維與假設檢定

本模組不是全新的開始，而是一次收束與展望：讓學員看清楚自己已站在哪裡，以及前方哪些路是真實可走的。

三小時結構：

時段	主題	時長
Part A	ML 前導：函數逼近的本質	60 分鐘
Part B	DL + Big Data 前導：規模與架構的轉折點	60 分鐘

Part C	課程收束 + 五條職涯路線地圖	60 分鐘

二、模組學習目標

完成本模組後，學員能夠：

1. 用「函數逼近」的語言解釋機器學習是什麼，而不是用魔法類比
2. 正確描述監督式學習的四個要素：特徵、目標、訓練集、測試集
3. 說明 scikit-learn 在 Python ML 生態中的定位，以及 1.0 版本的里程碑意義
4. 走完一個完整的建模工作流（分割 → 訓練 → 評估 → 迭代），而不是只知道 `.fit()`
5. 解釋深度學習為何在學完 NumPy / tensor 概念後更容易理解，而不是從零開始
6. 描述 PyTorch 2.0 的定位轉移：從「寫模型」到「編譯優化與效能工程」
7. 知道 pandas 的邊界在哪裡，以及 Arrow / Polars / DuckDB / PySpark 各自解決什麼規模的問題
8. 根據自身背景與興趣，選擇一條具體的後續學習路線並說出理由

三、投影片大綱

投影片總覽

#	標題	核心訊息	時段歸屬
S01	你已經走了多遠	回顧七個模組建立的底盤	Part A 開場
S02	ML 不是魔法，是函數逼近	解除「AI 神秘化」的心理屏障	Part A

S03	監督式學習的一張圖	X / y / train / test / feature / target	Part A
S04	scikit-learn：Python ML 的標準入口	里程碑 1.0 的意義	Part A
S05	建模工作流：不只是一行 `.fit()`	split → train → evaluate → iterate	Part A
S06	動手做：scikit-learn 分類 / 迴歸實作	在熟悉資料集上跑完完整流程	Part A 收束
S07	為什麼深度學習在這裡順理成章	tensor / autograd 已在前面出現過	Part B 開場
S08	PyTorch 2.0：從寫模型到效能工程	`torch.compile()` 的轉折	Part B
S09	當 pandas 不夠用時	Arrow → Polars / DuckDB → PySpark 的規模梯度	Part B
S10	PySpark：Python 進入大數據分散式的橋樑	分散式執行模型直覺	Part B 收束
S11	五條職涯路線地圖	統計 / ML / DL / DE / LLM 工程	Part C
S12	課程終章：你搭了一條底盤	最終心法與鼓勵	Part C 收束

S01：你已經走了多遠

核心訊息：
在進入前導內容之前，先讓學員看見自己已建立的資產，而不是只看到前方有多少還沒學。

講師要點：

- 前七個模組不是在「學 Python」，而是在建立一套資料分析的共同語言：型別、陣列、表格、視覺化、統計直覺
- 這些能力之間彼此相連：pandas 的 DataFrame 背後是 NumPy array；NumPy array 的概念直接對應 PyTorch tensor
- 你現在能做的事包含：從原始資料到清理、從清理到分析、從分析到洞察、從洞察到視覺化呈現——這本
- 本模組的任務不是「把這些也教完」，而是讓你知道「接下來要走的路，底盤已經在這裡了」

視覺建議：
一張學習路線圖，七個模組節點已點亮，第七個模組節點（本模組）位於圖的末端，後方分叉出五條路線箭頭，

過渡： 第一站，打開 ML 的黑盒子。

S02：ML 不是魔法，是函數逼近

核心訊息：
機器學習的本質是從資料中找到一個函數，把輸入映射到可預測的輸出——沒有魔法，只有最佳化。

講師要點：

- 很多人對 ML 的第一個印象是「AI 自己學會了」，這個說法雖然直覺，但容易讓人把它神秘化
- 更精確的說法是：給定一批 (input, output) 配對，ML 在找一個函數 f，使得 f(input) 盡可能接近 output
- 這個「找函數」的過程，就是訓練（training）；用來衡量「接近程度」的指標，就是損失函數（loss）
- 訓練結束後，我們用沒看過的資料（測試集）來確認這個函數能不能推廣（generalize），而不只是記住
- 從這個角度來看，線性迴歸、決策樹、神經網路，本質上都在做同一件事——只是函數的形式和複雜度不

視覺建議： 一個最簡化的示意圖：左側是一批散點（X, y），右側是一條曲線（模型）；箭頭標示「訓練」的方向；旁邊附一個公式欄，只寫 `f(X)` ≈

y，加上損失函數的文字說明，不展開數學推導。

過渡： 知道了「在找什麼」，接下來看監督式學習的四個核心概念。

S03：監督式學習的一張圖

核心訊息：
一張圖包含學員需要知道的六個術語：特徵、目標、訓練集、測試集、輸入、輸出。理解這張圖，就能讀懂 90% 的 ML 文章。

講師要點：

- 特徵（feature / X）：輸入給模型的資料欄位，例如「年齡、收入、消費頻率」
- 目標（target / label / y）：希望模型預測的值，例如「是否流失（分類）」或「下個月消費金額（迴歸）」
- 訓練集（train set）：拿來讓模型「看」並調整參數的資料，通常佔總資料的 70–80%
- 測試集（test set）：模型「沒看過」的資料，用來評估真實泛化能力；測試集的表現才是我們真正關心的
- 常見錯誤：在測試集上做決策再重新訓練，等於「考試前先看考題」——結果看起來好，但模型其實不可
- pandas DataFrame 裡已經有 X 和 y 的概念：X 是你選的欄位，y 是你指定的目標欄位

視覺建議： 一張清晰的流程圖，左側是原始 DataFrame，中間用虛線切分成 train / test 兩塊，右側分別接到「訓練模型」和「評估模型」兩個方塊；六個術語用標籤箭頭標注在對應位置。

過渡： 有了概念圖，在進入工具之前，先看看業界怎麼組織一個完整的資料科學專案。

S03-B：CRISP-DM — 資料科學專案的業界標準流程

核心訊息： CRISP-DM（Cross-Industry Standard Process for Data Mining）是資料科學專案最廣泛使用的流程框架。它不是線性的，而是迭代循環的。

講師要點：

- CRISP-DM 六階段：

- Business Understanding（商業理解）：定義問題。「我們要預測什麼？為什麼？成功的標準是什麼？」— 85% 的資料專案失敗不是因為技術，而是因為問題定義不清
- Data Understanding（資料理解）：EDA。看資料長什麼樣、有什麼品質問題、和業務問題的關聯是什麼 — 這就是 M5 教你做的事
- Data Preparation（資料準備）：清理、轉換、特徵工程 — 這就是 M4 教你做的事，通常佔整個專案 60-80% 的時間
- Modeling（建模）：選模型、訓練、調參 — 接下來 S04-S06 要教的
- Evaluation（評估）：模型好不好？不只是看數字，還要回到 Business Understanding 檢查：它真的解決了最初的問題嗎？
- Deployment（部署）：模型上線、監控、維護 — 屬於進階課程範疇
- 關鍵洞察：這不是直線，而是循環。評估結果可能讓你回到資料準備甚至商業理解重新定義問題
- 回顧本課程：M1-M6 其實已經走過了 CRISP-DM 的前三個階段。M7 是第四和第五階段的入門

視覺建議：CRISP-DM 的經典六角形循環圖，六個階段用箭頭連接成環形，中間標示「資料」。在每個階段旁邊標注對應的課程模組（對應 Business Understanding 思維、M4 對應 Data Preparation、M5 對應 Data Understanding 等）。

過渡：知道了完整流程，現在聚焦在建模這一步——需要一個工具把概念變成程式碼。

S04：scikit-learn：Python ML 的標準入口

核心訊息：scikit-learn 是 Python 傳統機器學習（非深度學習）的標準實作，1.0 版本的發布是這個生態成熟度的里程碑。

講師要點：

- scikit-learn 誕生於 2007 年，最初是 Google Summer of Code 專案，現在是全球最廣泛使用的 ML 函式庫之一

- 版本里程碑：scikit-learn 1.0（2021 年）代表這個函式庫正式進入穩定期，API 設計被視為業界標準參考
- 核心設計哲學：一致的 API 介面——分類器、迴歸器、聚類器全部遵循 `.fit()` / `.predict()` / `.score()` 三個方法，學一個等於學會框架
- scikit-learn 的邊界很清楚：它處理結構化資料（表格）的傳統 ML，不處理影像、語音、序列這類深度學習場景——邊界清楚是優點，不是缺陷
- 對學過 pandas 的你來說，scikit-learn 的輸入就是 DataFrame 或 NumPy array，沒有新的資料概念需要學

視覺建議： 一個「Python ML 生態地圖」，中央是 scikit-learn，周圍標示它的上游（NumPy / pandas）和下游（特徵工程、模型選擇、管線化），右側單獨標示「scikit-learn 1.0 里程碑：2021 年」的時間軸。

過渡： 有了工具，看完整的建模流程——不只是 `.fit()` 這一行。

S05：建模工作流：不只是一行 `.fit()`

核心訊息： 建模是一個迭代循環，包含分割資料、訓練、評估、診斷、調整五個步驟。只知道 `.fit()` 是最危險的半吊子狀態。

講師要點：

- 步驟一：分割資料——`train_test_split()` 是第一行，不是最後一行。分割的策略（比例、隨機種子、stratify）本身就是工程決策
- 步驟二：訓練模型——`.fit(X_train, y_train)`，這一行很短，但前面的特徵選擇、缺值處理、標準化才是大工作
- 步驟三：評估模型——`.score()` 或 `classification_report()`，看的是測試集，不是訓練集；訓練集表現好但測試集差 = 過擬合（overfitting）
- 步驟四：診斷與迭代——評估結果告訴你去哪裡：更多資料？換模型？調超參數？回頭看特徵工程？這是
- 建模不是「一次跑對」的過程，而是「每輪評估驅動下一輪改進」的工程循環——這個心態才是正確的

視覺建議：
一個循環箭頭示意圖，四個節點分別標示「分割」「訓練」「評估」「診斷/調整」，並用虛線標示「如果評估通
→
部署」的出口；在「診斷」節點下方列出三個分支：「過擬合」「欠擬合」「資料問題」，各自指向對應的處理

過渡： 概念講完，動手做一次。

S06：動手做：scikit-learn 分類 / 迴歸實作

核心訊息： 在熟悉的資料集上跑完完整的建模工作流，從 DataFrame 到評估報告，感受每一步的實際意義。

講師要點：

- 使用前幾個模組用過的真實資料集（例如客戶資料或公開資料集），而非使用 iris 或 titanic——熟悉的資料讓學員聚焦在流程而非資料理解
- 走完的程式碼骨架：`X, y = df[features], df[target] X_train, X_test, y_train, y_test = train_test_split(X, y) model = LogisticRegression() # ■ LinearRegression() model.fit(X_train, y_train) print(model.score(X_test, y_test)) print(classification_report(y_test, model.predict(X_test)))`
- 重點觀察：訓練集 score vs 測試集 score 的差距，讓學員親眼看見過擬合的現象
- 延伸挑戰（選做）：換一個模型（RandomForest），比較結果；試著調整 `test_size` 比例，觀察對評估指標的影響
- 這個練習的目的不是得到最好的模型，而是把工作流走熟一次

視覺建議： 在投影片中嵌入一個 Jupyter Notebook 截圖，顯示完整的程式碼骨架和一個真實的 `classification_report` 輸出；用紅框標示「測試集評估」的那幾行，強調這是最重要的輸出。

過渡： 傳統 ML 走完了。為什麼接下來深度學習對你來說不是從頭開始？

S07：為什麼深度學習在這裡順理成章

核心訊息： 深度學習不是一門全新的學科，它的基礎概念——tensor、矩陣乘法、梯度下降——你在 NumPy 和 PyTorch 的前導介紹中已經接觸過。

講師要點：

- 你已經知道的東西在 DL 裡叫什麼：NumPy array → tensor；矩陣乘法 → 前向傳播（forward pass）；找最小值 → 梯度下降（gradient descent）；`.grad` → autograd
- 深度學習的核心創新不是「發明了新數學」，而是「用多層函數組合，自動學習特徵」——這是對傳統 ML 手工特徵工程的替代
- 學傳統 ML 先，學 DL 後，這個順序是有意義的：因為 scikit-learn 的工作流（train/test split、評估、迭代）在 DL 裡完全一樣，只是模型結構不同
- 深度學習的門檻不在數學，在計算資源與資料量——你需要更大的資料集、GPU，以及更長的訓練時間；
- 最重要的直覺：DL 的「層」（layer）就是一系列函數的組合，每一層做一次「特徵變換」，多層堆疊讓模型能學到更複雜的模

視覺建議： 一個對照表，左欄是「你已學過的概念」，右欄是「在 DL 中的對應名稱」，共五行，用箭頭連接；下方附一個最簡單的神經網路示意圖（輸入層 → 隱藏層 × 2 → 輸出層），每層用方塊表示。

過渡： DL 的主流框架是 PyTorch——而 2.0 版本代表它進入了一個新的階段。

S08：PyTorch 2.0：從寫模型到效能工程

核心訊息： PyTorch 2.0 是深度學習框架的一個轉折點：核心創新 `torch.compile()` 讓 DL 從「只要能跑」進入「要跑得快、要在生產環境穩定」的工程化階段。

講師要點：

- PyTorch 2.0 里程碑（2023 年）：這不只是版本號的升級，而是框架設計哲學的轉移
- 過去的 PyTorch：以「動態計算圖」為核心優勢——Debug 方便、研究靈活，但生產部署效能相對弱
- `torch.compile()`：在保留動態圖靈活性的前提下，加入編譯式優化——在 A100 GPU 上的實測加速可達 30–200%，且不需要修改模型結構

- 這代表什麼轉變：DL 工程師的職責不再只是「寫出能收斂的模型」，也包括「讓模型在生產環境中跑得快、跑得穩」
- 對初學者的意義：現在學 PyTorch，不只是學 `nn.Linear` / `nn.ReLU`，而是要有「這段程式碼未來要跑在什麼環境」的效能直覺。系統層面的細節（GPU 執行模型、分散式訓練架構）在進階課程「系統設計與架構思維」中有完整介紹

視覺建議： 一個時間軸，標示 PyTorch 各主要版本的里程碑（1.0 穩定 API → 1.8 分散式 → 2.0 compile 優化）；旁邊附一個最小的程式碼對比：「PyTorch 1.x 的訓練迴圈」vs「加上 `torch.compile()` 的 2.0 版本」，只改一行，效能差異用數字標示。

過渡： ML 和 DL 都在處理「建模」問題。但如果資料本身的規模就超出單機記憶體，連建模前的預處理都跑不完，怎麼辦？

S09：當 pandas 不夠用時

核心訊息： pandas 是單機記憶體內的工具，當資料量超過機器 RAM，或需要高效能 SQL 式查詢，需要不同工具——Arrow / Polars / DuckDB / PySpark 是對應不同規模的解法。

講師要點：

- pandas 的邊界：實測上，超過記憶體 30–50% 的 DataFrame 就會開始變慢甚至 OOM（Out of Memory）；這不是 pandas 的缺陷，而是它的設計邊界
- Apache Arrow：一個記憶體格式標準，讓不同工具之間的資料交換不需要複製——pandas 2.0 的 Arrow backend 讓它在某些場景快上數倍
- Polars / DuckDB：同樣在單機運行，但用更現代的執行引擎（lazy evaluation、多核並行、向量化查詢），可以處理遠超 pandas 容量的資料，且語法更接近 SQL
- PySpark：當資料需要分散到多台機器才能處理時，PySpark 是 Python 進入分散式計算（Hadoop / Spark 生態）的橋樑
- 選哪個工具不是品味問題，是資料規模問題：100MB 用 pandas、10GB 用 Polars/DuckDB、100GB+ 用 PySpark——這是工程判斷，不是偏好

視覺建議： 一個「資料規模梯度」示意圖，X 軸是資料量（MB → GB → TB），Y 軸是工具，每個工具對應一個有效範圍的色塊：pandas（< 1GB）、Polars/DuckDB（1GB–100GB）、PySpark（100GB+）；圖下方加一句說明：「沒有最好的工具，只有

過渡： 在這個梯度的末端，PySpark 是 Python 工程師進入大數據世界的主要橋樑。

S10：PySpark：Python 進入大數據分散式的橋樑

核心訊息： PySpark 讓 Python 工程師能用熟悉的 DataFrame API 操作分散在多台機器上的資料，是資料工程與大數據分析職涯的關鍵入口。

講師要點：

- Apache Spark 是大數據分散式計算的主流框架，PySpark 是它的 Python API——你用 Python 寫，Spark 把工作分派到整個叢集
- 最重要的直覺：PySpark 的 DataFrame API 在語法上和 pandas 非常相似，但執行模型完全不同——pandas 是立即執行（eager），Spark 是延遲執行（lazy），只有在觸發 action 時才真正計算
- 這個「lazy evaluation」設計是有意義的：Spark 的優化器（Catalyst）在你下達所有指令後，一次性規劃最有效率的執行計畫，而不是一行一行跑
- 進入 PySpark 的前置條件：理解 Python、熟悉 DataFrame 概念、有基本的 SQL 查詢直覺——你在本課程全部建立了
- PySpark 背後連接著整個 Hadoop 生態（HDFS、Hive、YARN）和雲端資料倉儲（BigQuery、Databricks、AWS EMR），進入這裡等於進入企業級資料工程的核心。分散式執行模型的硬體層細節，在進階課程「系統設計

視覺建議： 一個分散式執行示意圖，左側是「Driver（你的 Python 程式）」，右側是一個叢集方塊，內含三個 Worker 節點，Driver 和叢集之間有雙向箭頭；圖下方用兩個方塊對比「pandas（單機，立即執行）」vs「PySpark（分散式，延遲執

過渡： 技術前導結束。現在你站在五條路的交叉口——讓我們把每條路說清楚。

S11：五條職涯路線地圖

核心訊息：
這門課的底盤通往五條不同的技術路線，每條路都是真實的職涯選項，選擇哪條取決於你對「資料」的哪個面向

講師要點：

- 這五條路線不是等級高低的排序，而是方向的分叉——沒有哪條「更好」，只有哪條更適合你現在的目標
- 每條路線都有清楚的「起點技能」（本課程已建立）和「下一步要學的東西」（課程結束後的方向）
- 最重要的決策依據：你對什麼問題感到好奇？是「為什麼這個模型比那個好？」還是「資料從哪裡來、怎 LLM 幫我做任務？」——問題的方向就是路線的方向
- 不必現在決定，但要現在想：完成這門課的第一週，試著選一條路，找一個入門專案動手做

五條路線詳細說明：

路線	方向	核心技能堆疊	適合誰
Route A	統計分析	機率、抽樣分布、假設檢定、迴歸分析、A/B test 設計與解讀	對「推論」與「因果判斷」感興趣；想 BI / 策略分析 / 產品數據分析
Route B	機器學習工程	scikit-learn、特徵工程、模型評估指標、交叉驗證、集成學習（Ra Forest, XGBoost）	對「建立預測系統」感興趣；想進 DS / ML Engineer / 推薦系統
Route C	深度學習	線性代數、tensor 計算圖、PyTorch、MLP / CNN / RNN / Transformer	對「學習表示」感興趣；想進 CV / NLP / 研究導向職位
Route D	資料工程	SQL、PySpark、ETL pipeline、Arrow / DuckDB / Polars、workflow orchestration（Airflow / Prefect）	對「資料如何流動、如何可靠地移動」 DE / Data Platform / MLOps 基礎設施

Route E	AI 應用 / LLM 工程	向量資料庫、embedding 概念、prompt workflow、RAG（Retrieval-Augmented Generation）、agent pipeline	對「讓 LLM 解決真實任務」感興趣；想進 AI 產品工程 / LLM Ops / GenAI 應用開發

路線選擇的判斷問題：

「你最常問的問題是哪種類型的？」

- 為什麼結果是這樣？（Route A：統計分析）
- 能不能預測得更準？（Route B：ML 工程）
- 機器怎麼看懂這張圖 / 這句話？（Route C：深度學習）
- 資料怎麼可靠地從 A 流到 B？（Route D：資料工程）
- 怎麼讓 AI 幫我完成這個任務？（Route E：LLM 應用）

視覺建議：
一張「路線地圖」示意圖，中央是一個「課程畢業點」圓圈，五條帶箭頭的路線從圓圈輻射出去，每條路線旁列

過渡： 路線地圖確定了。最後，讓我們把這 24 小時走過的東西說清楚。

S12：課程終章：你搭了一條底盤

核心訊息：
你沒有只學一套工具，你建立了一個底盤——讓你能在五條路上繼續走下去，而不需要從頭開始的那種底盤。

講師要點：

- 課程回顧：這門課從第一天就圍繞著一個核心目標——讓你能獨立完成從原始資料到有意義洞察的完整工

- 里程碑式進步：從「能讀 CSV」到「能清理資料」到「能做統計分析」到「能建一個分類模型」到「知道當資料規模大了要換什麼工具
- 心法一：「先懂資料，再懂模型；先搭底盤，再碰 AI。」——這個順序不是保守，而是讓你後面走得更快的投資
- 心法二：「You didn't learn a tool. You built a foundation.」——工具會換，但資料思維、統計直覺、程式設計能力不會換。你學的是那個不換的部分
- 下一步建議：選一條路線，找一個真實問題，用你現在的底盤去碰它。碰壁了，你會知道哪裡需要補——

視覺建議： 一張「Before / After」對比投影片，左側列出「開始前你知道什麼」（Python 基礎、可能一點點 pandas），右側列出「現在你能做什麼」（完整 EDA 流程、統計分析、ML 建模工作流、大數據工具選型判斷、五條職涯路線的第一步）；最下方單獨一行大字引用最終心法：

> "Machine learning turns patterns into prediction. You just learned how to prepare the patterns."

過渡： 課程結束。但你的學習才剛剛開始。

四、動手練習區

Part A 實作練習：完整建模工作流

目標： 在一個熟悉的資料集上走完一次完整的 scikit-learn 建模流程，從 DataFrame 到評估報告。

練習骨架：

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

# ═══════════════════════════════
# Step 1: 載入資料
df = pd.read_csv("data/customer_data.csv")

# Step 2: 定義特徵
features = ["age", "monthly_spend", "tenure_months", "num_products"]
target = "churn"
```

```
# 
X = df[features].dropna()
y = df.loc[X.index, target]
# 
# Step 3: 
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
# 
# Step 4: 
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# 
# Step 5: 
model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)
# 
# Step 6: 
train_score = model.score(X_train_scaled, y_train)
test_score = model.score(X_test_scaled, y_test)
# 
print(f"Train accuracy: {train_score:.4f}")
print(f"Test  accuracy: {test_score:.4f}")
print()
print(classification_report(y_test, model.predict(X_test_scaled)))
```

觀察重點： 1. `train_score` 和 `test_score` 的差距有多大？超過 5% 就值得懷疑是否過擬合 2. `classification_report` 中，precision 和 recall 哪個對你的問題更重要？ 3. 試著把 `LogisticRegression` 換成 `RandomForestClassifier`，結果如何變化？

延伸挑戰（選做）：

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score
import numpy as np
# 
# 5-fold cross validation
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
cv_scores = cross_val_score(rf_model, X, y, cv=5, scoring="f1")
# 
print(f"CV F1 scores: {cv_scores}")
print(f"Mean F1: {np.mean(cv_scores):.4f} (+/- {np.std(cv_scores):.4f})")
```

Part B 實作練習：工具規模判斷

目標： 根據資料大小，選擇正確的工具，並比較執行時間。

```
import pandas as pd
import time

# ═══════════════════
def benchmark_pandas(filepath: str) -> float:
    start = time.time()
    df = pd.read_csv(filepath)
    result = df.groupby("category")["value"].sum()
    return time.time() - start

# ══════ polars
import polars as pl
def benchmark_polars(filepath: str) -> float:
    start = time.time()
    df = pl.scan_csv(filepath).group_by("category").agg(pl.col("value").sum()).collect()
    return time.time() - start

# ═══
1. ═══════════ pandas ═══
2. ═══════════ 100 ═══════
3. ═══════ 50 ═══ HDFS ═══
```

Part C 反思練習：職涯路線選擇

這不是程式練習，是思考練習。

用一張紙（或一個 Notion 頁面），回答以下三個問題：

> 我最感到好奇的問題是哪種類型的？ 把你的答案對應到五條路線中的一條。
>
> 我的下一個 30 天學習計畫是什麼？
> 根據選擇的路線，列出三個具體的行動（例如：讀某本書的第 X 章、做某個 Kaggle 競賽、複製某個開源專案的一部分）。
>
> 我的「不知道」清單是什麼？
> 把你現在知道自己不懂的東西列出來，這比「知道什麼」更有價值——因為它是你的學習地圖。

五、五條職涯路線詳細說明

Route A：統計分析路線

核心問題：「這個現象是真實的，還是隨機誤差造成的？」

技術堆疊（按學習順序）：

層次	技術內容
基礎統計	機率分布（常態、二項、泊松）、期望值、變異數
推論統計	抽樣分布、中央極限定理、信賴區間
假設檢定	t-test、chi-square test、ANOVA、p-value 的正確解讀
迴歸分析	線性迴歸診斷、多元迴歸、交互效應、殘差分析
實驗設計	A/B test 設計（樣本數計算、顯著水準、統計功效）
應用工具	Python `scipy.stats`、`statsmodels`、R（選修）

典型職位： 商業分析師、策略分析師、產品數據分析師、BI 工程師、研究分析師

本課程已建立的前置底盤： pandas 資料清理（M4）、統計基礎（M6）、視覺化呈現（M5）

Route B：機器學習工程路線

核心問題：「我能不能讓預測系統在真實環境中可靠地運行？」

技術堆疊（按學習順序）：

層次	技術內容

特徵工程	缺值處理策略、類別編碼（one-hot、target encoding）、特徵縮放
模型評估	ROC-AUC、PR curve、F1 score、confusion matrix 解讀
交叉驗證	k-fold、stratified fold、time-series split
集成學習	Random Forest、Gradient Boosting（XGBoost、LightGBM）
超參數調整	Grid Search、Random Search、Bayesian Optimization
管線化	scikit-learn Pipeline、ColumnTransformer
模型部署基礎	joblib 序列化、FastAPI 包裝、Docker 打包

典型職位：機器學習工程師、資料科學家、推薦系統工程師、風險模型工程師

本課程已建立的前置底盤：scikit-learn 工作流（本模組 Part A）、pandas 特徵處理（M4）、統計評估直覺（M6）

Route C：深度學習路線

核心問題：「機器如何從原始資料中自動學到有用的表示？」

技術堆疊（按學習順序）：

層次	技術內容
數學基礎	線性代數（矩陣乘法、特徵值）、微積分（鏈式法則）、機率論

Tensor 與 Autograd	PyTorch tensor 操作、計算圖、`.backward()` 機制
基礎架構	全連接層（MLP）、激活函數、損失函數、優化器（Adam、SGD）
卷積神經網路	CNN 結構、影像特徵提取、ResNet / EfficientNet
序列模型	RNN、LSTM、GRU、時序資料建模
Transformer	注意力機制、self-attention、BERT / GPT 架構直覺
效能工程	`torch.compile()`、混合精度訓練、分散式訓練概念

典型職位： 深度學習工程師、電腦視覺工程師、NLP 工程師、AI 研究員

本課程已建立的前置底盤： NumPy tensor 概念（M3）、PyTorch 前導（本模組 Part B）、Python 基礎（M2）

Route D：資料工程路線

核心問題：「資料怎麼可靠地從原始來源流動到可用的狀態？」

技術堆疊（按學習順序）：

層次	技術內容
SQL 基礎	SELECT / JOIN / GROUP BY / Window Functions、查詢優化直覺
分散式計算	PySpark DataFrame API、RDD 概念、執行計畫（explain()）
ETL 設計	資料管線設計原則、idempotency、錯誤重試、資料品質驗證

現代資料工具	Apache Arrow、DuckDB（嵌入式 OLAP）、Polars（Rust-based DataFrame）
工作流編排	Apache Airflow DAG 設計、Prefect、任務依賴管理
資料格式	Parquet、ORC、Avro、Delta Lake、Iceberg
雲端資料倉儲	BigQuery / Snowflake / Redshift 基礎查詢模式

典型職位： 資料工程師、Data Platform 工程師、MLOps 工程師、Analytics Engineer

本課程已建立的前置底盤： pandas 資料處理（M4）、PySpark 前導（本模組 Part B）、Python 基礎（M2）

> 資料管線的系統底層設計（process 模型、I/O 架構、分散式儲存原理）在進階課程「系統設計與架構思維」中有完整介紹。

Route E：AI 應用 / LLM 工程路線

核心問題：「怎麼讓大型語言模型可靠地解決真實的業務問題？」

技術堆疊（按學習順序）：

層次	技術內容
Embedding 基礎	向量表示的直覺、相似度計算（cosine similarity）、embedding 模型
向量資料庫	Chroma / Pinecone / Weaviate 的基本操作、索引與查詢
Prompt 工程	系統提示設計、few-shot learning、prompt 模板管理

RAG 架構	Retrieval-Augmented Generation 流程設計、文件分塊策略
LLM API 整合	OpenAI / Claude / Gemini API 使用、token 計算與成本控制
Agent 管線	Tool use、function calling、multi-step 推理流程
評估與監控	LLM 輸出評估指標、幻覺（hallucination）偵測、生產監控

典型職位： AI 應用工程師、LLM Ops 工程師、GenAI 產品工程師、Conversational AI 工程師

本課程已建立的前置底盤： Python 基礎（M2）、pandas 資料處理（M4）、統計思維（M6）

六、課程收束

單一路線回顧

這 24 小時的課程以「讓初學者能獨立完成資料分析工作流」為核心目標，一步一步往前推進：

```
M0 [redacted]
[redacted]
M1 [redacted]
[redacted]
M2 Python [redacted]
[redacted]
M3 NumPy [redacted] tensor [redacted]
[redacted]
M4 pandas [redacted] EDA [redacted]
[redacted]
M5 [redacted]
[redacted]
M6 [redacted]
[redacted]
M7 ML [redacted]
[redacted]
[redacted] +-- Route A: [redacted]
[redacted] +-- Route B: [redacted]
[redacted] +-- Route C: [redacted]
```

```
+-- Route D: [redacted]
+-- Route E: AI [redacted] / LLM [redacted]
```

每一個模組都建立在前一個之上，也為後面打開了新的可能性：pandas 的 DataFrame 背後是 NumPy array；統計思維讓你知道模型評估指標的意義；視覺化讓你能診斷資料問題而不只是跑程式碼。

最終心法

> 先懂資料，再懂模型；先搭底盤，再碰 AI。

這個順序不是在叫你慢下來，而是讓你後面走得更快的投資邏輯。很多人在碰 AI 工具的時候卡住，不是因為 AI 太難，而是底盤還沒搭穩。

這門課做的事情只有一件：幫你把底盤搭穩。

> "You didn't learn a tool. You built a foundation."

工具會換，框架會更新，API 會改版，但資料思維、統計直覺、程式設計能力不會換。你在這 24 小時裡學的，是那個不換的部分。

參考文獻與引用來源

機器學習框架

1. scikit-learn Official Documentation — Python 傳統 ML 的標準入口。始於 2007 年 David Cournapeau 的 Google Summer of Code 專案，1.0 版於 2021 年 9 月 24 日發布。scikit-learn.org
2. scikit-learn 1.0 Release Highlights — scikit-learn.org/stable/auto_examples/release_highlights/plot_release_highlights_1_0_0.html

深度學習

1. PyTorch 2.0 Release Blog (March 2023) — `torch.compile()` 在 163 個開源模型上測試，93% 成功編譯，A100 GPU 上平均訓練加速 43%（Float32 精度 21%、AMP 精度 51%）。pytorch.org/blog/pytorch-2-0-release
2. PyTorch Official Documentation — pytorch.org/docs

CRISP-DM 框架

1. Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C. & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step Data Mining Guide. — CRISP-DM 的原始規格文件，由 NCR、DaimlerChrysler、SPSS、OHRA 聯合制定。kde.cs.uni-kassel.de/lehre/ws2012-13/kdd/files/CRISPWP-0800.pdf
2. Wirth, R. & Hipp, J. (2000). "CRISP-DM: Towards a Standard Process Model for Data Mining." — CRISP-DM 的學術論文。

大數據工具

1. PySpark Official Documentation — Python 操作 Apache Spark 分散式運算。spark.apache.org/docs/latest/api/python
2. Polars Documentation — 高效能 DataFrame 引擎。docs.pola.rs
3. DuckDB Documentation — 嵌入式 OLAP 資料庫。duckdb.org/docs
4. Apache Arrow — 跨語言欄式記憶體格式。arrow.apache.org

產業數據

1. 「85% 的大數據專案失敗」 — 原始來源：Gartner 分析師 Nick Heudecker 於 2017 年表示 Gartner 最初估計的 60% 失敗率「太保守」，實際接近 85%。另見 VentureBeat (2019) 報導：87% 的資料科學專案未能進入生產環境。參見 designingforanalytics.com/resources/failure-rates-for-analytics-bi-iot-and-big-data-projects-85-yikes
2. 「資料準備佔專案 60-80% 時間」 — CrowdFlower (2016). Data Science Report：約 60% 時間用於清理，19% 用於收集。此數據在 Forbes、HBR 等被廣泛引用。Kaggle (2018) State of Data Science Survey 報告為 15-25%。實際比例因組織成熟度而異。

3. 「訓練/測試分割通常為 70-80% / 20-30%」 — 此為 ML 社群廣泛接受的經驗法則。Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning, 2nd Edition. Springer. 第七章討論了交叉驗證與資料分割策略。免費版本：hastie.su.domains/ElemStatLearn

延伸閱讀

1. Hastie, T., Tibshirani, R. & Friedman, J. (2009). The Elements of Statistical Learning, 2nd Edition. Springer. — ML 理論的標準教科書。
2. Geron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 3rd Edition. O'Reilly Media. — 實務導向的 ML 入門書。

下一步

1. 選一條路線——不用現在就確定一輩子，只要選一個 30 天的方向
2. 找一個真實問題——不是教材上的資料集，是你自己遇到的、想解決的問題
3. 用你的底盤去碰它——碰壁了，你會知道哪裡需要補；那才是真正的學習起點

> "Machine learning turns patterns into prediction. You just learned how to prepare the patterns."

模組資訊

項目	內容
模組編號	M7（最終模組）
時長	3 小時
課程位置	24 小時課程第 22–24 小時

前置模組	M0–M6（全部模組）
後續方向	五條職涯路線（Route A–E）